

Linux & Shell Mastery

Complete Reference — MIT Missing Semester & Beyond

A comprehensive reference covering Linux filesystem internals, shell mechanics, scripting, and every command tool needed for daily terminal work and research cluster use.

19 sections · Filesystem · Permissions · Shell · Globs · Regex · Redirections · Scripting · ls · find · grep · sed · awk · sort/uniq · xargs · curl · Essential Utilities · JSON & jq · Exercise Takeaways · Cheatsheet

1. Linux Filesystem Internals

1.1 Disk Layout (ext4)

A disk is a flat sequence of fixed-size **blocks** (typically 4 KB). ext4 divides the disk into **block groups**, each containing:

- **Superblock** — filesystem metadata: total inodes/blocks, block size, UUID, mount time, filesystem state (clean/dirty for fsck). Backup copies in multiple block groups.
- **Block bitmap & Inode bitmap** — one bit per slot (0=free, 1=used). rm flips the block bitmap — instant regardless of file size.
- **Inode table** — array of inodes for this block group.
- **Data blocks** — actual file/directory contents.

1.2 Inodes

Every file and directory has exactly one **inode**. It stores everything *about* the file — but NOT its name:

Flag / Syntax	Description
File type + permissions	The 10-character permission string
UID / GID	Numeric owner/group IDs — resolved to names via /etc/passwd, /etc/group
File size	Content bytes. For directories: metadata size (~4096), NOT total contents
mtime / ctime / atime	Modification, metadata-change, and access timestamps
Hard link count	Number of directory entries pointing to this inode
Data block pointers	Which blocks on disk hold the actual content

Note: The filename is NOT stored in the inode. Names live only in directory entries.

1.3 Directory Entries & Path Resolution

A directory is a file whose data blocks contain a **name→inode number hashmap**. Path resolution walks this chain:

```
For /home/safwan/notes.txt:
start at inode 2 (hardcoded root /)
→ look up "home" in root dir → inode 48
→ look up "safwan" in inode 48 → inode 391
→ look up "notes.txt" in 391 → inode 5821
→ permission check → read data blocks
```

Relative paths start from the CWD inode stored **per-process** in kernel memory. **cd** is a **shell builtin** (not /usr/bin/cd) because a child process cannot change its parent's CWD.

1.4 Hard Links vs Symbolic Links

Property	Hard Link	Symbolic Link
What it points to	Inode number (direct)	A path string (stored as file data)
Separate inode?	No — shares original inode	Yes — its own inode, type l
Target deleted	Still works (inode count > 0)	Dangling — path resolves to nothing

Property	Hard Link	Symbolic Link
Cross filesystems	Impossible — inode# only per-fs	Works — just a string
Link to directory	Not allowed (prevents cycles)	Works fine (common use)
Create with	In target linkname	In -s target linkname

Note: Every filename IS a hard link. `rm` is called `unlink` — removes one directory entry, decrements link count. Data freed only when count=0 AND no process has an open file descriptor to it.

1.5 Mounting & Partitions

Linux has ONE unified tree starting at `/`. Partitions are grafted onto the tree at **mount points** — not separate islands like Windows `C:/D/`.

```
/dev/nvme0n1p1 → / (root partition)
/dev/nvme0n1p2 → /home (home partition)
/dev/sdb1 → /mnt/data (external/NTFS)
```

- **lsblk** — disk/partition tree with mount points
- **df -h** — mounted filesystems with usage
- **/etc/fstab** — persistent mount config read at boot

1.6 ext4 vs NTFS

Property	ext4	NTFS
Metadata	Inode table	Master File Table (MFT)
Filename	In directory entry only	Inside MFT record itself
Small files	Separate data blocks	Inline in MFT (resident)
Directories	Flat hash table	B-tree index (faster for huge dirs)
Permissions	Unix DAC: UID/GID + rwx bits	Windows ACLs
Hard links	Clean, multiple dir entries	Supported but limited
Linux access	Native	Via ntfs3 kernel driver

2. Linux Permissions

2.1 The 10-Character Permission String

```

d r w x r - x r - x
^ |-----| |-----| |-----|
| owner group other
file type

```

File type character (position 1)

Flag / Symbol	Description
-	Regular file
d	Directory
l	Symbolic link (ls shows name -> target)
c	Character device — byte-stream: keyboard, terminal (lives in /dev)
b	Block device — block-addressable: disks, USB (lives in /dev)
p	Named pipe (FIFO) — created with mkfifo
s	Socket — bidirectional IPC (Docker, PostgreSQL)

Permission characters (rwx)

Char	On File	On Directory
r	Read file contents	List directory contents (ls)
w	Modify file contents	Create/delete files inside
x	Execute as program	Enter directory (cd), access files inside
-	Not granted	Not granted
s	Setuid/setgid — run as owner	Run as file group
t	Rarely used on files	Sticky — only owner can delete (e.g. /tmp)

Note: Kernel checks owner → group → other in ORDER and STOPS at first match. Owner permissions --- deny access even if group has rwx.

2.2 chmod / chown / chgrp

```

chmod 755 file # rwxr-xr-x (octal)
chmod +x script.sh # add execute for all
chmod u+w,g-w file # symbolic: add owner-write, remove group-write
chmod -R 644 dir/ # recursive
chown safwan file # change owner
chown safwan:dev file # change owner AND group
chgrp dev file # change group only
ls -ln file # show raw UID/GID numbers (no name resolution)

```

2.3 Timestamps (stored per inode)

Flag / Syntax	Description
mtime	Content modification time — ls -l (default), ls -lt to sort
ctime	Metadata change (permissions, owner, link count) — ls -lc
atime	Last access/read time — ls -lu

2.4 ACLs

A + at end of permission string means extended ACL entries exist.

```
getfacl filename # view ACL entries
setfacl -m u:alice:rw filename # give alice read+write
setfacl -m g:devteam:r filename # give devteam read only
```

3. Shell Fundamentals

3.1 Expansion Order

1. Brace expansion `{a,b,c}` `{1..5}`
2. Tilde expansion `~` `→ /home/safwan`
3. Variable expansion `$VAR` `${VAR}`
4. Command substitution `$(cmd)` ``cmd``
5. Arithmetic expansion `$(2+3)`
6. Word splitting `spaces/tabs/newlines` `→ argument boundaries`
7. Glob expansion `* ? [...]`
8. Program runs

3.2 Special Characters

Flag / Syntax	Description
<code>\$</code>	Variable/command/arithmetic expansion; special params
<code>`</code>	Old-style command substitution — prefer <code>\$()</code> instead
<code>* ? []</code>	Glob pattern matching against filenames
<code>{ }</code>	Brace expansion or command grouping
<code>~</code>	Home directory expansion
<code>> < >></code>	Redirection operators
<code> </code>	Pipe — connect stdout to stdin of next command
<code>&</code>	Background (<code>&</code>) or logical AND (<code>&&</code>)
<code>;</code>	Command separator — run next regardless of exit code
<code>()</code>	Subshell — run commands in child shell
<code>!</code>	History expansion (interactive) / NOT in <code>[[]]</code>
<code>#</code>	Comment — rest of line ignored
<code>\</code>	Escape next character literally
<code>' "</code>	Quoting mechanisms
space/tab/newline	Word splitting — separates arguments

3.3 Quoting

What expands?	Single <code>' '</code>	Double <code>" "</code>	ANSI-C <code>\$' '</code>
<code>\$VAR</code> expansion	No	Yes	No
<code>\$(cmd)</code> substitution	No	Yes	No
<code>\$(n)</code> arithmetic	No	Yes	No
<code>!</code> history	No	Yes — avoid	No
Glob <code>* ? []</code>	No	No	No

What expands?	Single ' '	Double " "	ANSI-C \$' '
<code>\n \t</code> escapes	No — literal	No — literal	Yes — actual char
Single quote inside	Impossible directly	Works	Use <code>'</code>

```

echo 'Literal: $HOME ! *' # nothing expands
echo "Hello $USER, $((2+2))" # variable + arithmetic expand
echo $'Tab:\there\nNewline' # actual tab and newline
echo 'it'\''s here' # single quote inside single-quoted string
# Print $, newline, ! in one string:
echo '$'$'\n'!'

```

3.4 Dollar-Sign Operators

Flag / Syntax	Description
<code>\$VAR</code> or <code>\${VAR}</code>	Variable expansion
<code>\$(...)</code>	Command substitution — replaced by command output
<code>\$((...))</code>	Arithmetic expansion
<code>\$'...'</code>	ANSI-C quoting — interprets <code>\n \t \x41 \uNNNN</code>
<code>\$?</code>	Exit code of last command (0=success)
<code>\$#</code>	Number of positional arguments
<code>"\$@"</code>	All args as separate quoted words (prefer over <code>\$*</code>)
<code>"\$*"</code>	All args as one string (joined by IFS first char)
<code>\$0</code>	Script's own name
<code>\$1 ... \$9</code>	Positional args (use <code>\${10}</code> for double-digit indices)
<code>\$\$</code>	PID of current process
<code>\$!</code>	PID of last background job

3.5 String Parameter Expansion

Powerful built-in string manipulation — no external command needed:

Flag / Syntax	Description
<code>\${var:-default}</code>	Use default if var is unset or empty
<code>\${var:=default}</code>	Set var to default if unset, then use it
<code>\${#var}</code>	Length of string
<code>\${var%.*}</code>	Remove shortest suffix matching pattern (e.g. strip extension)
<code>\${var%%.*}</code>	Remove longest suffix matching pattern
<code>\${var#*/}</code>	Remove shortest prefix matching pattern
<code>\${var##*/}</code>	Remove longest prefix — get filename from path
<code>\${var/old/new}</code>	Replace first occurrence of old with new
<code>\${var//old/new}</code>	Replace all occurrences

Flag / Syntax	Description
<code>\${var^}</code>	Uppercase first character
<code>\${var^^}</code>	Uppercase entire string
<code>\${var,,}</code>	Lowercase first character
<code>\${var,,,}</code>	Lowercase entire string

```
file="/home/safwan/model.pt"
echo "${file##*/}" # → model.pt (filename only)
echo "${file%.*}" # → /home/safwan/model (no extension)
echo "${file##*.}" # → pt (extension only)

name="hello"
echo "${name^^}" # → HELLO
echo "${#name}" # → 5
```


4. Glob Patterns

Pattern	Meaning	Example
*	Any sequence of chars (not . at start)	ls *.py
?	Exactly ONE character	ls file?.txt
[abc]	Any one char in set	ls file[123].txt
[a-z]	Any one char in range	ls [a-z]*.sh
[^abc]	Any char NOT in set	ls .[^.]*
{a,b}	Brace expansion — generates strings (no fs check)	touch {a,b,c}.txt
{1..5}	Brace range expansion	echo {1..5}
**	Recursive depth (needs shopt -s globstar)	ls **/*.py

Note: Globs match filename *COMPONENTS* not full paths. * never crosses /. * does NOT match dotfiles by default. [] checks filesystem (only existing files); {} generates strings regardless of filesystem.

Quoting Rule — Who Expands the Pattern?

```
ls *.py # shell expands — correct for current dir
find . -name "*.py" # quoted — find does its own matching as it traverses
echo *.py # debug: shows exactly what shell expands
```

Practical Examples

```
# Create project structure
mkdir -p project/{src,data,logs,models}

# Copy multiple configs
cp configs/{base,dev,prod}.yaml /backup/

# Match single-digit versions only
ls model_v[0-9].py

# List dotfiles without recursing into hidden dirs
ls -d .*
```

5. Regular Expressions

5.1 Metacharacters

Flag / Syntax	Description
.	Any single character (except newline)
*	Zero or more of preceding element
+	One or more of preceding (ERE/PCRE)
?	Zero or one of preceding (ERE/PCRE)
{n,m}	Between n and m of preceding
^	Start of line anchor
\$	End of line anchor
[abc]	Any one char in set
[^abc]	Any char NOT in set
[a-z]	Any char in range
(...)	Capture group — saves for \1 backreference
	Alternation — OR
\b	Word boundary
\d	Digit [0-9] (PCRE only)
\w	Word char [a-zA-Z0-9_] (PCRE only)
\s	Whitespace (PCRE only)
\.	Literal dot (escaped metacharacter)
&	In sed replacement: entire match

5.2 Flavours

Flavour	Where used	Meta-chars	Lookaheads
BRE (Basic)	grep, sed default	\+ \? \ \ (\)	No
ERE (Extended)	grep -E, sed -E, awk	+ ? ()	No
PCRE	grep -P, Python, JS	+ ? ()	Yes — (?=...) (?<=...)

5.3 Greedy Matching

.^{*} is greedy — matches as much as possible, then backtracks. So `.*\.` finds the LAST dot, not the first. Lookaheads `(?=...)` are PCRE only — not available in `grep -E` or `sed -E`.

5.4 Common Patterns

```
^Error # line starts with Error
\.py$ # line ends with .py
[0-9]+ # one or more digits
^$ # empty line
\.[^./]+$ # file extension (dot + non-dot-non-slash + end)
^[a-zA-Z_][a-zA-Z0-9_]*$ # valid identifier
([0-9]{4})-([0-9]{2})-([0-9]{2}) # date with capture groups
```

6. Redirections, File Descriptors & Pipes

6.1 File Descriptors

```
0 → stdin (input)
1 → stdout (normal output)
2 → stderr (error output)
3+ → opened by program (next available integer)
```

6.2 Redirection Operators

Flag / Syntax	Description
cmd > file	stdout to file (overwrite)
cmd >> file	stdout to file (append)
cmd 2> file	stderr to file
cmd &> file	both stdout and stderr to file (bash shorthand)
cmd > file 2>&1	stderr → wherever stdout points (MUST be this order)
cmd 2>/dev/null	discard stderr
cmd >/dev/null 2>&1	discard all output
cmd tee file	stdout to terminal AND file simultaneously
cmd < file	file → stdin
cmd << EOF ... EOF	here-document — multiline string as stdin
cmd <<< 'str'	here-string — single string as stdin

Note: ORDER MATTERS: `cmd 2>&1 > file` is **WRONG** (stderr→terminal). Correct: `cmd > file 2>&1` — shell reads left to right, stdout must redirect first.

6.3 Pipes

- | creates an anonymous pipe — kernel buffer (~64 KB), no filesystem entry.
- **Both commands run concurrently** — left fills buffer, right drains it simultaneously.
- **Backpressure** — left pauses when buffer fills; resumes when right drains.
- **Named pipe (FIFO)** — created with `mkfifo`, has a filename, type `p` in `ls -l`.

```
python train.py | tee training.log # see output AND save
yes | head -n 5 # yes killed when head closes its end
```

6.4 Process Substitution

Treats command output as a file — useful for `diff` and tools that need file arguments:

```
diff <(sort file1) <(sort file2) # diff sorted versions without temp files
while IFS= read -r line; do # iterate lines of command output safely
echo "$line"
done <<(find . -name "*.py")
```

7. Shell Scripting

7.1 Shebang & Standard Header

```
#!/usr/bin/env bash # PREFERRED: finds bash via $PATH (portable)
#!/bin/bash # explicit path (less portable)
set -euo pipefail # safety: exit-on-error + undefined-var + pipe-failure
```

7.2 set Flags

Flag / Syntax	Description
-e	Exit immediately on any non-zero exit code
-u	Treat undefined variables as errors
-o pipefail	Pipe fails if ANY command in it fails (not just last)
-x	Xtrace: print each command before running (shows actual values)
-v	Verbose: print source lines as read (before expansion)
-n	Noexec: parse but don't execute — syntax check (bash -n script.sh)
-f	Noglob: disable all glob expansion
-C	Noclobber: prevent > from overwriting existing files
+x	TURN OFF xtrace (+ disables, - enables — counterintuitive!)

7.3 Conditionals — [[]] vs []

Always prefer [[]] — bash keyword with own parser (no word splitting). [] is a regular command.

```
# File tests
[[ -f "$f" ]] -d -e -L -r -w -x -s # same in both

# String tests
[[ -z "$s" ]] # empty [[ -n "$s" ]] # non-empty
[[ "$a" == "$b" ]] # equal [[ "$a" != "$b" ]] # not equal
[[ "$f" == *.py ]] # glob match — RHS MUST be unquoted
[[ "$s" =~ ^[0-9]+$ ]] # regex match — RHS MUST be unquoted

# Numeric — ALWAYS use these for numbers (never > < which are string compare)
[[ "$a" -eq "$b" ]] -ne -lt -gt -le -ge

# Arithmetic context — natural math syntax
(( a >= 5 )) (( a++ )) (( result = a * b ))

# Combining
[[ -f "$f" && -r "$f" ]] # AND
[[ -f "$f" || -d "$f" ]] # OR
[[ ! -f "$f" ]] # NOT
```

Note: > and < inside [[]] are LEXICOGRAPHIC — "9" > "10" is TRUE as strings. Use -gt -lt for numeric comparison, or (()) arithmetic context.

7.4 if / elif / else

```
#!/usr/bin/env bash
set -euo pipefail

if [[ -z "$1" ]]; then
echo "Usage: $0 <filename>"; exit 1
elif [[ -f "$1" ]]; then
echo "$1 is a file"
elif [[ -d "$1" ]]; then
echo "$1 is a directory"
else
echo "$1 does not exist"
fi
```

7.5 Loops

```
# for loop
for f in *.py; do echo "Processing $f"; done

# while loop
while [[ $i -lt 10 ]]; do ((i++)); done

# Iterate command output safely (handles spaces in lines)
while IFS= read -r line; do
echo "$line"
done < file.txt

# C-style for
for ((i=0; i<10; i++)); do echo $i; done
```

7.6 Positional Params vs Environment Variables

Property	Positional	Environment
Set by	Caller on command line	OS/shell or export
Access	<code>\$1 \$2 ... @\$#</code>	<code>\$VARNAME</code>
Scope	One script invocation	Inherited by all child processes
Typical	Script inputs (config, file)	Runtime config (GPU, paths, tokens)

```
export CUDA_VISIBLE_DEVICES=0,1 # env var: available to all children
bash train.sh configs/exp.yaml # $1 = "configs/exp.yaml"

# Pass shell vars safely (never interpolate directly into awk/jq)
awk -v t="$threshold" '$2 > t'
jq --arg m "$MODEL" '.[] | select(.name == $m)'
```

7.7 Functions & Exit Codes

```
function greet() {
local name="$1" # local: only exists inside function
echo "Hello, $name"
return 0
}
greet "Safwan"
```

```
cmd && echo "success" || echo "failed" # && on exit 0, || on non-zero
cmd1 ; cmd2 # always runs cmd2
echo $? # last exit code
```

7.8 Bash Arrays

```
arr=("model_a" "model_b" "model_c")
echo "${arr[0]}" # first element
echo "${arr[@]}" # all elements (separately quoted)
echo "${#arr[@]}" # array length

# Iterate
for item in "${arr[@]}"; do echo "$item"; done

# Append
arr+=("model_d")
```

7.9 Job Control

```
cmd & # run in background
jobs # list background jobs
fg %1 # bring job 1 to foreground
bg %1 # resume stopped job 1 in background
Ctrl+Z # suspend current foreground job
Ctrl+C # kill current foreground job
nohup cmd & # run cmd immune to hangup (survives SSH disconnect)
disown %1 # detach job from shell (no HUP signal on exit)
```

8. ls — List Directory Contents

Flags

Flag / Syntax	Description
-l	Long format — shows all 7 metadata fields
-a	Show all including hidden dotfiles
-h	Human-readable sizes — REQUIRES -l to have any effect
-t	Sort by modification time, newest first
-r	Reverse sort order
-i	Show inode number as first column
-n	Show raw UID/GID numbers instead of names
-c	Show ctime instead of mtime
-u	Show atime instead of mtime
-d	Show directory itself not contents (e.g. ls -ld /tmp)
-lah	Most common combo — long + all + human-readable
-tr	Sort oldest-first (reverse time sort)

The -l Output Format — All 7 Fields

```
drwxr-xr-x 2 safwan safwan 4096 Jan 5 12:00 src/  
1 2 3 4 5 6 7
```

1. Permissions (10 chars)
2. Hard link count
3. Owner (username)
4. Group name
5. Size in bytes (dirs: metadata size ~4096)
6. Last modified (mtime)
7. Filename

Note: Use `du -sh dirname/` for actual disk usage of all contents — not `ls`.

9. find — Search the Filesystem

9.1 Test Flags

Flag / Syntax	Description
-type f	Regular files only
-type d	Directories only
-type l	Symbolic links only
-name "*.py"	Filename matches glob (case-sensitive) — MUST quote
-iname "*.py"	Case-insensitive name match
-path "*/src/*"	Full path matches glob
-mtime +7	Modified MORE than 7 days ago (older)
-mtime -7	Modified LESS than 7 days ago (within last week)
-mtime 7	Modified exactly 7 days ago
-mmin -60	Modified within last 60 minutes
-newer file	mtime newer than reference file
-newermt "date"	mtime newer than literal date string
-not -newer f	mtime older than or equal to reference file
-size +100M	Larger than 100 MB (k=KB M=MB G=GB)
-empty	Empty files or directories
-perm -u+x	Has execute bit for owner
-user name	Owned by specific user
-maxdepth 1	Only current directory, no subdirectories
-mindepth 2	Skip first N levels
-not or !	Negate: find . -not -name "*.py"
-o	OR — always group: \(-name "*.log" -o -name "*.tmp" \)

9.2 Action Flags

Flag / Syntax	Description
-print0	Null-terminated output — pair with xargs -0 (handles spaces)
-printf "%f\n"	%f=filename, %p=full path, %s=size, %T+=timestamp
-delete	Delete matched files (careful — no confirmation)
-exec cmd {} \;	Run cmd ONCE PER FILE — {} replaced by filename
-exec cmd {} +	Batch ALL files into ONE call (faster; {} must be last)
-execdir cmd {} \;	Like -exec but runs from the file's own directory

Flag / Syntax	Description
<code>-prune</code>	Skip traversal INTO matched dirs (more efficient than <code>-not -path</code>)

Note: `-exec {} +` requires `{}` as LAST arg. `-prune` skips traversal; `-not -path` filters after traversal. Always group `-o`: `\( cond1 -o cond2 \) -action`

9.3 Practical Examples

```
# Python files modified today, >1KB
find . -type f -name "*.py" -mtime -1 -size +1k

# Delete __pycache__ dirs
find . -type d -name "__pycache__" -exec rm -rf {} +

# Large files eating disk quota
find ~ -type f -size +100M -printf "%s %p\n" | sort -rn | head -20

# Exclude noise dirs efficiently
find . -type d \( -name __pycache__ -o -name .git \) -prune \
-o -type f -name "*.py" -print

# Safe with spaces in filenames
find . -name "*.sh" -print0 | xargs -0 wc -l

# Files produced during training run
find . -newer training_started.marker -type f
```

10. grep — Search Text

Flags

Flag / Syntax	Description
-E	Extended regex (ERE) — enables + ? () without backslash
-P	Perl regex (PCRE) — enables lookaheads, \d \w \s etc.
-i	Case-insensitive
-v	Invert — print lines that do NOT match
-o	Print only the matching part (not whole line)
-n	Show line numbers
-r/-R	Recursive — search all files in directory tree
-l	Print only filenames of files with matches
-c	Count matching lines
-e pat	Add a pattern (allows multiple: -e "err" -e "warn")
-f file	Read patterns from file (one per line)
-A n	Print n lines After each match
-B n	Print n lines Before each match
-C n	Print n lines before AND after (combined context)
-w	Match whole words only
-q	Quiet — exit 0 if found, 1 if not (use in scripts with &&)

Examples

```
grep "Error" *.log # search multiple files
grep -rn "TODO" src/ # recursive with line numbers
grep -oE "\.[^./]+$" paths.txt # extract file extensions
grep -v "^#" config.txt # skip comment lines
grep -e "error" -e "warning" app.log # multiple patterns
grep -q "pattern" file && echo "found" # silent check for scripts
```

11. sed — Stream Editor

11.1 Flags

Flag / Syntax	Description
-E	Extended regex — + ? () without backslash
-n	Suppress automatic printing (use with p command)
-i	Edit file in-place
-i.bak	In-place with backup copy
-e 'cmd'	Add expression (for chaining commands)
-f file	Read sed script from file

11.2 Substitution

```
s/pattern/replacement/ # replace first match per line
s/pattern/replacement/g # replace ALL matches (global)
s/pattern/replacement/i # case-insensitive

# Change delimiter (when pattern contains /)
s|old/path|new/path|g

# Capture groups and backreferences
sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\3\/\2\/\1/'
# YYYY-MM-DD → DD/MM/YYYY

# & = entire match
sed 's/[0-9]*/[&]/g' # 42 → [42]
```

11.3 Address Syntax

```
sed '3s/foo/bar/' # only line 3
sed '1,5s/foo/bar/' # lines 1-5
sed '/error/s/old/new/' # lines containing error
sed '/start/,/end/s/x/y/' # lines between patterns
sed '/^#!/s/foo/bar/' # all EXCEPT comment lines
```

11.4 Other Commands

Flag / Syntax	Description
d	Delete lines: sed '/^#/d'
p	Print (use with -n): sed -n '/error/p' file
q	Quit: sed '10q' file (like head -10)
=	Print line number: sed -n '/error/= ' file

11.5 Practical Examples

```
# In-place config editing (most important use)
sed -i 's/lr=0.01/lr=0.001/' config.txt
sed -i 's/batch_size=[0-9]*/batch_size=64/' config.txt

# Remove blank lines and comments
sed '/^$/d; /^#/d' config.txt

# Extract username from SSH log (Missing Semester pipeline)
sed -E 's/.*user (.*?) [^ ]+ port.*\/1/' ssh.log

# Replace across all Python files
sed -i 's/old_func/new_func/g' *.py
```

12. awk — Column & Text Processor

12.1 Execution Model

```
awk 'pattern { action }' file

BEGIN { } # once before any input
{ } # once per line (no pattern = every line)
END { } # once after all input
/regex/ { } # lines matching regex
$2 > 100 { } # lines where field 2 > 100
```

12.2 Built-in Variables

Flag / Syntax	Description
\$0	Entire current line
\$1 ... \$NF	Individual fields (\$NF = last field)
NF	Number of fields on current line
NR	Line number (across all files)
FNR	Line number within current file only
FS	Field separator (default: whitespace)
OFS	Output field separator (default: space)
FILENAME	Name of current input file

12.3 Flags

Flag / Syntax	Description
-F", "	Comma-separated input
-F'\t'	Tab-separated input
-v t=100	Pass shell variable in as \$t (CORRECT way)
-f prog	Read awk program from file

12.4 Actions & Field Manipulation

```
{ print $1, $3 } # comma → OFS between fields
{ printf "%-10s %5.2f\n", $1, $2 } # C-style formatting

{ $1 = "Z"; print } # replace first field
{ $4 = ""; print } # clear fourth field
{ tmp=$1; $1=$3; $3=tmp; print } # swap fields
{ $1=$1; print } # rebuild $0 with new OFS

/^#/ { next } # skip comment lines
NR > 1000 { exit } # stop processing
```

12.5 Arrays

```
# Frequency count
awk '{ count[$1]++ } END { for (k in count) print count[k], k }' file

# Sum per group
awk '{ sum[$1] += $2 } END { for (k in sum) print k, sum[k] }' file

# Deduplicate (preserve order)
awk '!seen[$0]++'
```

12.6 String Functions

Flag / Syntax	Description
length(\$1)	Character count
substr(\$1, 2, 4)	Substring (pos 2, length 4)
gsub(/old/, "new")	Replace ALL in \$0
gsub(/o/, "n", \$2)	Replace ALL in \$2 only
sub(/old/, "new")	Replace FIRST in \$0
tolower(\$1)	Lowercase
toupper(\$1)	Uppercase
split(\$1,a,":")	Split into array a by :, returns count
sprintf("%05d",\$2)	Format without printing

12.7 Practical Examples

```
# Average column 3 (skip header)
awk 'NR > 1 { sum += $3; n++ } END { print sum/n }' metrics.csv

# Swap fields 1 and 3 where col 2 > 100 (Missing Semester exercise)
printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 { tmp=$1; $1=$3; $3=tmp; print }'

# Count error types
awk '/ERROR/ { count[$4]++ } END { for (t in count) print count[t], t }' app.log | sort -rn

# Reformat CSV to TSV
awk -F',' 'BEGIN{OFS="\t"} {$1=$1; print}' data.csv

# Available memory
awk '/MemAvailable/ { print $2/1024, "MB" }' /proc/meminfo
```

13. sort / uniq / wc / head / tail / cut / paste / tr

13.1 sort

Flag / Syntax	Description
-n	Numeric sort (42 > 8 correctly)
-r	Reverse order
-rn	Numeric + reverse — highest first (most common for counts)
-k1,1	Sort by column 1 only
-k2,2n	Sort by column 2 numerically
-t,	Use comma as field separator
-u	Unique — remove duplicates after sorting

13.2 uniq

Flag / Syntax	Description
-c	Prefix each unique line with occurrence count
-d	Print only lines that appear more than once
-u	Print only lines that appear exactly once

Note: *uniq only detects CONSECUTIVE duplicates. Always sort first: sort file | uniq -c | sort -rn*

13.3 wc / head / tail

Flag / Syntax	Description
wc -l	Count lines
wc -w	Count words
wc -c	Count bytes
head -n 5	First 5 lines
tail -n 5	Last 5 lines
tail -f	Follow — stream new lines live (monitor training logs)
tail -n +2	Skip first line — start from line 2 (skip CSV headers)

13.4 cut

Flag / Syntax	Description
cut -d, -f2	Extract field 2 with comma delimiter
cut -d: -f1	Extract field 1 with colon delimiter
cut -c1-10	Extract characters 1-10 from each line

Flag / Syntax	Description
<code>cut -d, -f1,3</code>	Extract fields 1 and 3

13.5 paste

Flag / Syntax	Description
<code>paste -sd,</code>	Join all lines into comma-separated string
<code>paste f1 f2</code>	Merge two files side by side, tab-separated
<code>paste -d\t f1 f2</code>	Use tab as column delimiter

```
echo -e "alice\nbob\ncharlie" | paste -sd, # → alice,bob,charlie
```

13.6 tr — Translate/Delete Characters

Flag / Syntax	Description
<code>tr a-z A-Z</code>	Uppercase all lowercase letters
<code>tr -d '\r'</code>	Remove carriage returns (Windows line endings)
<code>tr -s ' '</code>	Squeeze multiple spaces into one
<code>tr -d '[:punct:]'</code>	Remove all punctuation
<code>tr ',' '\n'</code>	Replace commas with newlines (one item per line)

```
# Fix Windows line endings on cluster files
tr -d '\r' < windows_file.txt > unix_file.txt

# Count words (alternative)
echo "hello world foo" | tr ' ' '\n' | wc -l
```

13.7 The Frequency Pipeline

```
# Template: find most common X
source | sort | uniq -c | sort -rn | head -n10

# Most common file extensions
find ~ -type f | grep -oE "\.[^./]+$" | sort | uniq -c | sort -rn | head -5

# Most used commands from history
grep -oE '[a-z]+' ~/.bash_history | sort | uniq -c | sort -rn | head -10
```

14. xargs — Build and Execute Commands

14.1 Flags

Flag / Syntax	Description
-0	Read null-separated input — pair with find -print0 (handles spaces)
-n 1	ONE argument per invocation
-P 4	Run 4 parallel processes (huge speedup for batch preprocessing)
-I {}	Placeholder: replace {} with argument (can appear anywhere, multiple times)
-t	Print each command before running (debugging)
-r	Don't run command if stdin is empty

14.2 Examples

```
# Safe (handles filenames with spaces)
find . -name "*.sh" -print0 | xargs -0 wc -l

# Parallel image preprocessing
find data/ -name "*.png" | xargs -P 8 -I {} python resize.py {}

# Backup all yaml configs
find . -name "*.yaml" -print0 | xargs -0 -I {} cp {} {}.bak

# Give all scripts execute permission
find scripts/ -name "*.sh" -print0 | xargs -0 chmod +x
```

15. curl — Transfer Data from URLs

15.1 Flags

Flag / Syntax	Description
-sSL	Standard combo: silent + show errors + follow redirects
-o filename	Save to specific filename
-O	Save with remote filename
-C -	Resume interrupted download (auto-detect offset)
--limit-rate 5M	Limit bandwidth
-H "Key: Val"	Add request header (auth, content-type)
-X POST	Specify HTTP method (default: GET)
-d '{"k": "v"}'	Send request body (POST data)
-f	Fail on HTTP errors — returns non-zero for && logic
-w "%{http_code}"	Print response metadata after transfer
-I	Fetch headers only (HEAD request)
-v	Verbose — show full request/response (debugging)

15.2 Examples

```
# HuggingFace API with token
curl -sSL -H "Authorization: Bearer $HF_TOKEN" \
https://huggingface.co/api/models/bert-base-uncased | jq .

# POST JSON to inference server
curl -sSL -X POST -H "Content-Type: application/json" \
-d '{"prompt": "hello"}' http://localhost:8080/generate

# Health check in script
curl -sSf URL > /dev/null && echo "up" || echo "down"

# Download with resume
curl -C - -sSL https://example.com/data.tar.gz -o data.tar.gz

# Verify integrity after download
sha256sum -c dataset.tar.gz.sha256

# SECURITY: save before running pipe-to-shell installs
curl -sSL https://install.tool.com | tee install.sh | bash
```

16. Essential Utilities

16.1 date

Flag / Syntax	Description
%Y	Full year: 2026
%m	Month: 06
%d	Day: 01
%H	Hour (24h): 14
%M	Minute: 30
%F	Full date = %Y-%m-%d
%T	Full time = %H:%M:%S
%s	Unix timestamp (seconds since 1970)

```
cp model.pt "model_$(date +%F).pt" # → model_2026-06-01.pt
LOGFILE="run_$(date +%s).log" # unique log filename
```

16.2 tee

```
python train.py | tee training.log # see AND save
python train.py | tee -a training.log # append mode
```

16.3 tar — Archive & Compress (Critical for Datasets)

Flag / Syntax	Description
tar -czf out.tar.gz dir/	Create compressed archive (gzip)
tar -cjf out.tar.bz2 dir/	Create compressed archive (bzip2)
tar -xzf file.tar.gz	Extract gzip archive
tar -xf file.tar.gz -C /dest/	Extract to specific directory
tar -tzf file.tar.gz	List contents without extracting
tar -czf - dir/ ssh host "tar -xzf - && rm -rf /tmp/dest"	Stream archive over SSH

```
# Download and extract in one command (common for datasets)
curl -sSL https://example.com/data.tar.gz | tar -xzf -
```

16.4 rsync — File Transfer to Clusters

Flag / Syntax	Description
rsync -avz src/ user@host:dest	Sync local to remote (verbose, archive, compress)
rsync -avz user@host:src/ dest	Sync remote to local
rsync -avz --exclude="__pycache__" src/ user@host:dest	Exclude patterns

Flag / Syntax	Description
<code>rsync -avz --dry-run src/ host:dest/</code>	Test what would be transferred
<code>rsync -avzP src/ host:dest/</code>	Show progress per file (-P = --partial --progress)

Note: *rsync is better than scp for syncing — it only transfers changed files. Use -avzP for large dataset transfers to research clusters.*

16.5 du / df — Disk Usage

Flag / Syntax	Description
<code>du -sh dir/</code>	Human-readable total size of directory
<code>du -sh *</code>	Size of each item in current directory
<code>du -h --max-depth=1</code>	One level deep breakdown
<code>df -h</code>	All mounted filesystems with usage
<code>df -h .</code>	Filesystem of current directory

```
# Find what is eating your cluster quota
du -sh * | sort -rh | head -20

# Check if you have space before downloading
df -h /scratch
```

16.6 less — Read Files Interactively

Flag / Syntax	Description
<code>less file</code>	Open file for reading (q to quit)
<code>less +G file</code>	Open at end of file
<code>less +F file</code>	Follow mode (like tail -f but interactive)
<code>/ pattern</code>	Search forward (n=next, N=previous)
<code>G / g</code>	Go to end / beginning
<code>q</code>	Quit

16.7 watch — Repeat a Command

Flag / Syntax	Description
<code>watch -n 2 cmd</code>	Run cmd every 2 seconds
<code>watch nvidia-smi</code>	Monitor GPU usage during training
<code>watch -n 5 "df -h"</code>	Watch disk usage
<code>watch -d cmd</code>	Highlight changes between runs

16.8 which / type — Find Commands

```
which python # full path of python executable
which -a python # all python executables in PATH
```

```
type ls # shows if builtin, alias, or external
type cd # → cd is a shell builtin
type ll # → ll is aliased to ...
```

16.9 Aliases & .bashrc

```
# Add to ~/.bashrc
alias ll='ls -lah'
alias gs='git status'
alias python='python3'

# Reload .bashrc without restarting shell
source ~/.bashrc
# or
. ~/.bashrc

# Make alias persistent: add to ~/.bashrc not just run in terminal
```

16.10 ssh — Remote Command Pattern

```
# Run command on remote server, pipe output back locally
ssh myserver 'journalctl -u sshd -b-1 | grep Disconnected' | sed ...

# Copy file to cluster
scp localfile user@cluster:/path/to/dest

# The quoted command executes ON THE SERVER
# Its stdout comes back over SSH to your local pipeline
```

16.11 nohup — Survive SSH Disconnect

```
nohup python train.py > train.log 2>&1 &
# nohup: immune to SIGHUP (hangup when SSH disconnects)
# &: run in background
# > train.log 2>&1: redirect all output to log

# Better alternative: tmux or screen (then detach)
# tmux new -s training
# python train.py
# Ctrl+B then D to detach (job keeps running)
```

17. JSON & jq

17.1 JSON Rules

- 6 types: string (double-quotes only), number, boolean (true/false), null, array [], object {}
- No comments, no trailing commas, no single-quoted strings
- **JSONL** — one JSON object per line — critical for LLM training data (streaming without loading full file)

17.2 jq Mental Model

jq is a functional assembly line: data enters left, flows through filters connected by |. No explicit loops — iteration by exploding with [].

- . always means current context at this point in the pipeline
- .key navigates into a field of current object
- .[] EXPLODES array/object — destroys container, puts each item on stream
- select() gatekeeper — true: item passes, false: item vaporised

17.3 Navigation & Iteration

```
. # identity: entire input
.key # field access
.[0] .[-1] # array index (0-based, negative from end)
.[2:5] # slice: elements 2,3,4
.a.b.c # chained navigation
.[ ] # iterator: explode array OR object values into stream
.key[] # shorthand for .key | .[ ]
```

17.4 Filtering & Construction

```
select(.version > 6) # filter by condition
select(has("error")) # filter by key existence
select(.val | type == "number" and . > 0) # inner pipe trick

map(f) # apply to every element, keep as array
map(select(.active)) # filter array
del(.key) # remove a key
{name: .name, v: .version} # build new object
.[ ] | .name # collect stream back into array
"\"(.name) scored \"(.score)\" # string interpolation
```

17.5 Aggregation

```
length # count elements/chars/keys
add # sum: [1,2,3] | add → 6
min_by(.loss) # object with minimum field — best checkpoint
max_by(.accuracy) # object with maximum field
sort_by(.epoch) # sort by field
group_by(.status) # group into sub-arrays
unique_by(.model) # deduplicate by field
```

17.6 Object Ops

```
keys # sorted array of keys
has("key") # boolean — key exists?
type # "string" "number" "array" "object" "boolean" "null"
to_entries # [{key:k, value:v}] — for key-value iteration
from_entries # inverse of to_entries

# TYPE ORDERING QUIRK: null < false < true < number < string < array < object
# ALWAYS check type before numeric comparison on mixed data:
.[ ] | select(.val | type == "number" and . > 5)
```

17.7 Variables & Logic

```
.max as $t | .items[] | select(.val > $t) # snapshot for later
if .loss > 10 then "EXPLODED" else "OK" end # else is MANDATORY
.name, .version # parallel streams
reduce .[] as $x (0; . + $x) # fold to single value
walk(if type == "object" then del(.debug) else . end) # recursive
```

17.8 Flags

Flag / Syntax	Description
-r	Raw output — no quotes on strings (pipe to shell)
-c	Compact — one line (for JSONL output)
-s	Slurp — ALL input into one array (critical for JSONL)
-n	Null input — construct JSON without reading stdin
--arg k v	Pass shell variable as jq string \$k (safe, avoids injection)
--argjson k v	Pass shell variable as JSON value

17.9 ML / Cluster Patterns

```
# Best checkpoint
jq "max_by(.accuracy) | .checkpoint_path" results.json

# Process JSONL: best epoch across all lines
cat epochs.jsonl | jq -s "min_by(.val_loss)"

# Export to CSV
jq -r ".runs[] | [.epoch, .loss, .accuracy] | @csv" log.json

# Pass shell variable safely
jq --arg m "$MODEL_NAME" ".results[] | select(.model == \"$m\")" results.json

# Extract names from HuggingFace API
curl -sSL "https://huggingface.co/api/models" | jq -r ".[].id"
```


18. Missing Semester Exercise Takeaways

Dotfile Listing

```
ls -d .* # -d prevents recursing into hidden subdirectories
# .[!..]* excludes . and .. but still recurses into .testdir
# -d flag is the real universal fix
```

Quoting — Print \$, Newline, ! in One String

```
echo '$$\n'!'
# '$' → literal $ (single quotes)
# '$\n' → actual newline (ANSI-C quotes)
# '!' → literal ! (single quotes)
# No single mechanism handles all three – combining is correct, not cheating
```

Redirection Exercise

```
ls /nonexistent /tmp > stdout.txt 2> error.txt # separate streams
ls /nonexistent /tmp &> both.txt # merge streams
```

Script Exercise — check.sh

```
# BUG: unquoted $1 disappears → [ -f ] → "-f" non-empty → truthy!
[ -f $1 ]

# FIX: quoted empty string → [ -f "" ] → correctly false
[[ -f "$1" ]]

# BEST: check for missing arg first
[[ -z "$1" ]] && { echo "Usage: $0 <file>"; exit 1; }
```

Flaky Test Script — @\$ vs \$1

```
# WRONG: $1 only passes first word
while cargo test my_test > "$LOGFILE" 2>&1; do ...

# CORRECT: "$@" passes all arguments as separate words
while "$@" > "$LOGFILE" 2>&1; do ...

# $! saves background job PID for later kill $STRESS_PID
# LOGFILE="run_$(date +%s).log" gives unique filename each run
```

File Extension Frequency

```
find ~ -type f | grep -oE "\.[^./]+$" | sort | uniq -c | sort -rn | head -5
# -o: extract only match, not whole line
# [^./]+: chars that are NOT dot or slash
# $: end of line – ensures last extension, not dots in path
# Two sorts needed: first groups, second orders by count
```

Most Used Commands from History

```
grep -oE '^[a-z]+' ~/.bash_history | sort | uniq -c | sort -rn | head -10
# ^[a-z]+: first word of line = base command name
```

SSH Log Pipeline

```
ssh myserver 'journalctl -u sshd -b-1 | grep Disconnected' \
| sed -E 's/.*user (.*?) [^ ]+ port.*\/\1/' \
| sort | uniq -c | sort -nk1,1 | tail -n10 \
| awk '{print $2}' | paste -sd,

# sed: (...) captures username, \1 in replacement outputs it
# sort -nk1,1: numeric sort on column 1 only
# paste -sd,: join all lines into comma-separated string
```

awk Column Swap Exercise

```
printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 { tmp=$1; $1=$3; $3=tmp; print }'
# Classic 3-step swap via tmp variable
# Field assignment forces awk to rebuild $0
# Pattern runs BEFORE action – a 50 x is filtered out
```

jq Exercise

```
curl -sS URL | jq -r "[]" | select(.version > 6) | .name"
# .[] → explode array into stream
# select → gatekeeper (vaporises non-matching)
# .name → extract field from survivors
# -r → raw output (no quotes)
```

curl + grep — Count Lectures

```
curl -sS https://missing.csail.mit.edu/ | grep -oE '<a href="/2026/[^"]+>' | wc -l
# [^"]+ requires at least one char after /2026/
# Excludes the bare /2026/ nav link – gives correct lecture count
```

19. Quick Reference Cheatsheet

Filesystem Facts

- inode stores: permissions, UID/GID, timestamps, size, block pointers — NOT filename
- Filename lives in directory entry (name→inode hashmap). Every filename IS a hard link.
- rm=unlink: removes directory entry, decrements count. Data freed when count=0 AND no open FDs.
- mv on same filesystem = instant: only updates directory entries. No data blocks moved.
- cp creates new inode + copies all data blocks. Hard link = new dir entry → same inode.
- Inode numbers unique per filesystem only — hard links cannot cross partitions.
- Symlink stores path string. Rename/delete target → symlink dangles.

One-Liners to Know Cold

```
#!/usr/bin/env bash
set -euo pipefail # ALWAYS start scripts with this

# Find Python files modified today, >1KB
find . -type f -name "*.py" -mtime -1 -size +1k

# Safe find + process (handles spaces in filenames)
find . -name "*.sh" -print0 | xargs -0 wc -l

# Frequency count template
source | sort | uniq -c | sort -rn | head -10

# Edit config in-place on cluster
sed -i 's/lr=0.01/lr=0.001/' config.txt

# Sum a column
awk '{ sum += $3 } END { print sum }' data.txt

# Timestamp a file
cp model.pt "model_$(date +%F).pt"

# Parallel preprocessing
find data/ -name "*.wav" | xargs -P 8 -I {} python process.py {}

# Best ML checkpoint
jq "max_by(.accuracy) | .path" results.json

# Download large dataset with resume
curl -C - -sSL https://example.com/data.tar.gz -o data.tar.gz

# Sync to cluster (only transfer changed files)
rsync -avzP --exclude="__pycache__" ./ user@cluster:/path/project/

# Run training job that survives SSH disconnect
nohup python train.py > train.log 2>&1 &

# Monitor GPU every 2 seconds
watch -n 2 nvidia-smi
```

```
# Get filename from path, strip extension
echo "${filepath##*/}" # filename only
echo "${filepath%.*}" # strip extension

# Swap fields 1 and 3 where col 2 > 100
awk ' $2 > 100 { tmp=$1; $1=$3; $3=tmp; print } '

# Count file extensions in home dir
find ~ -type f | grep -oE "\.[^./]+$" | sort | uniq -c | sort -rn | head -5

# Process JSONL: best epoch
cat runs.jsonl | jq -s "min_by(.val_loss)"
```

Tool Selection Guide

Task	Tool(s)
Find files by name/type/date/size	find
Search text in files	grep
Transform/replace text in lines	sed
Process structured columns/rows	awk
Process JSON data	jq
Fetch URLs / call APIs	curl
File transfer to cluster (efficient)	rsync
Archive/extract datasets	tar
Pass filenames to commands safely	xargs with -print0/-0
Sort and count frequencies	sort uniq -c sort -rn
Monitor live log output	tail -f or watch
Save output AND continue pipeline	tee
Edit files in-place	sed -i
Parallel batch processing	xargs -P
String manipulation (no subprocess)	\${var##*} \${var%.*} \${var^^}
Run jobs that survive disconnect	nohup cmd & or tmux
Compare checkpoints	jq max_by / min_by